



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: KARINA JEZULETH HERRERA
GALLEGOS

N° de Cuenta: 422526115

GRUPO DE LABORATORIO: 11

GRUPO DE TEORÍA: 04

SEMESTRE 2025-2

FECHA DE ENTREGA LÍMITE: 04 / 09 / 2025

CALIFICACIÓN: _____

Reporte de práctica 8: Iluminación 2

Se implementó un modelo de la corona del rey helado, por lo cual se tuvo que implementar tanto un model como su renderizado.

```
//Modelo corona  
Model corona;
```

```
//Modelos corona  
  
corona = Model();  
corona.LoadModel("Models/corona.obj");
```

Después de agregar dicho model, abajo se colocaron variables las cuales funcionan en primer lugar como booleanos para tener un control de lo que estará encendido y apagado, y en segundo se agregara una variable la cual me dara la opcion de saber si se aumenta o disminuye en el movimiento del carro.

```
//Bandera para la luz del carro (Determina si esta encendido una u otra luz)  
//Tambien se colocan variables para tener control de la distancia y el movimiento  
static double Datos = 0.0;  
static double Final = 0.0;  
bool luz_carro = false;
```

Se crean luces puntuales nuevas, tanto para la lámpara debido a que se usaron las spotlight para esta en la práctica anterior, como para la corona del rey helado.

```
//Declaracion de luz puntual Para la corona
pointLights[0] = PointLight(0.7f, 0.0f, 0.0f,
    0.6f, 0.6f,
    -0.2f, 1.5f, 16.5f,
    0.0f, 0.1f, 0.06f);
pointLightCount++;

//Declaración de luz puntual para la lampara
pointLights[1] = PointLight(1.0f, 1.0f, 1.0f,
    0.1f, 0.1f,
    -15.0f, 3.0f, 13.5f,
    0.0f, 0.1f, 0.06f);
pointLightCount++;
```

Se agregara una luz extra en el carro para la iluminación trasera, y para que se sepa si se aumenta o se disminuye en X la distancia recorrida se usan las variables mencionadas con anterioridad, con su respectiva bandera

```
Final = Datos;

Datos = mainWindow.getarticulacion7() + mainWindow.getarticulacion8();

if (Datos > Final) {
    Final = -5; // Si Decision es mayor que des, se resta des de Decision
}
else if (Datos < Final) {
    Final = 5; // Si Decision es menor que des, se resta Decision de des
}

// Actualizar el estado de la luz basado en la señal de articulacion3
if (Final == -5) {
    luz_carro = true;
}
else if (Final == 5) {
    luz_carro = false;
}

// Aplicar el estado actual de la luz
if (luz_carro) {
    spotLights[1].SetFlash(Faro, glm::vec3(0.0f, 0.0f, 0.0f));
    spotLights[3].SetFlash(Faro_trasero, glm::vec3(1.0f, 0.0f, 0.0f));
}
else {
    spotLights[1].SetFlash(Faro, glm::vec3(-1.0f, 0.0f, 0.0f));
    spotLights[3].SetFlash(Faro_trasero, glm::vec3(0.0f, 0.0f, 0.0f));
}
```

```

// Establece la posición base del capó
glm::vec3 capoPosition = glm::vec3(-5.4f, 2.5f, 2.0f);

// Obtén los valores de las articulaciones 7 y 8 (estos valores deberían obtenerse de métodos en 'mainWindow')
float desplazamientoX = mainWindow.getarticulacion7() + mainWindow.getarticulacion8(); // Suma los desplazamientos de las articulaciones en el eje X

// Actualiza la posición del capó sumando el desplazamiento calculado al valor de X
capoPosition.x += desplazamientoX; // La posición del capó se ajusta en el eje X según las articulaciones

// Dirección inicial de la luz (suponiendo que originalmente apunta hacia el eje X negativo)
glm::vec3 direccionLuzOriginal = glm::vec3(-1.0f, 0.0f, 0.0f);

// Aplica una rotación a la dirección de la luz basada en la rotación del modelo (en el eje Z)
glm::mat4 rotacionLuz = glm::rotate(glm::mat4(1.0f), glm::radians(-mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
glm::vec3 direccionLuzRotada = glm::vec3(rotacionLuz * glm::vec4(direccionLuzOriginal, 0.0f));

// Aplica la nueva posición y dirección de la luz al tercer spot light
spotLights[2].setFlash(capoPosition, direccionLuzRotada); // La luz se desplaza en el eje X y rota con el capó

```

Se usaron banderas en el mismo window.cpp, aunque no booleanas, si no que se utilizaron con valores de 1 y 0.

```

//Apagar y encender luz
if (key == GLFW_KEY_Z && action == GLFW_PRESS)
{
    static int estado = 1;

    // Cambia entre 0 y 1 cada vez que se presiona la tecla
    estado = 1 - estado;

    // Actualiza el valor de articulacion13
    theWindow->articulacion13 = estado;
}

//Apagar y encender luz
if (key == GLFW_KEY_X && action == GLFW_PRESS)
{
    static int estado = 1;

    // Cambia entre 0 y 1 cada vez que se presiona la tecla
    estado = 1 - estado;

    // Actualiza el valor de articulacion11
    theWindow->articulacion11 = estado;
}

```

Después de eso simplemente se agregó a la lámpara y a la corona del rey helado de la siguiente manera, los dos viéndose con un algoritmo similar.

```
//Llamamos la luz
// Verificar el valor de articulacion13 para encender o apagar la luz
if (mainWindow.getarticulacion13() == 1) {
    luz_PA = true;
}
else if (mainWindow.getarticulacion13() == 0) {
    luz_PA = false;
}

// Activar o desactivar la luz según su estado actual
if (luz_PA) {
    shaderList[0].SetPointLights(&pointLights[1], 1);
}
else {
    // La luz se apagada
    shaderList[0].SetPointLights(&pointLights[1], 0);
}
```

```
//Llamamos la luz
// Verificar el valor de articulacion11 para encender o apagar la luz
if (mainWindow.getarticulacion11() == 1) {
    luz_PA = true;
}
else if (mainWindow.getarticulacion11() == 0) {
    luz_PA = false;
}

// Activar o desactivar la luz según su estado actual
if (luz_PA) {
    shaderList[0].SetPointLights(&pointLights[0], 1);
}
else {
    // La luz se apagada
    shaderList[0].SetPointLights(&pointLights[0], 0);
}
```

Resultados





Conclusión

Tuve algunas dificultades al trabajar con los pointlight, principalmente por su naturaleza de no adherirse a las superficies como los spotlight, lo que dificultó su posicionamiento adecuado. Sin embargo, fuera de este inconveniente, no hubo mayores complicaciones. También, al renderizar el modelo de la corona, no se texturizaba de manera correcta.

Esta práctica me permitió aprender a mover luces mediante su enlace con piezas específicas del modelo. Además, aprendí a implementar un booleano en el archivo window.cpp para gestionar eventos, en lugar de hacerlo directamente en el código principal. Este enfoque facilitó la gestión de la interacción entre el sistema de luces y las piezas del modelo, haciendo la experiencia más dinámica.